Daniel Hernandez

CSCI-611

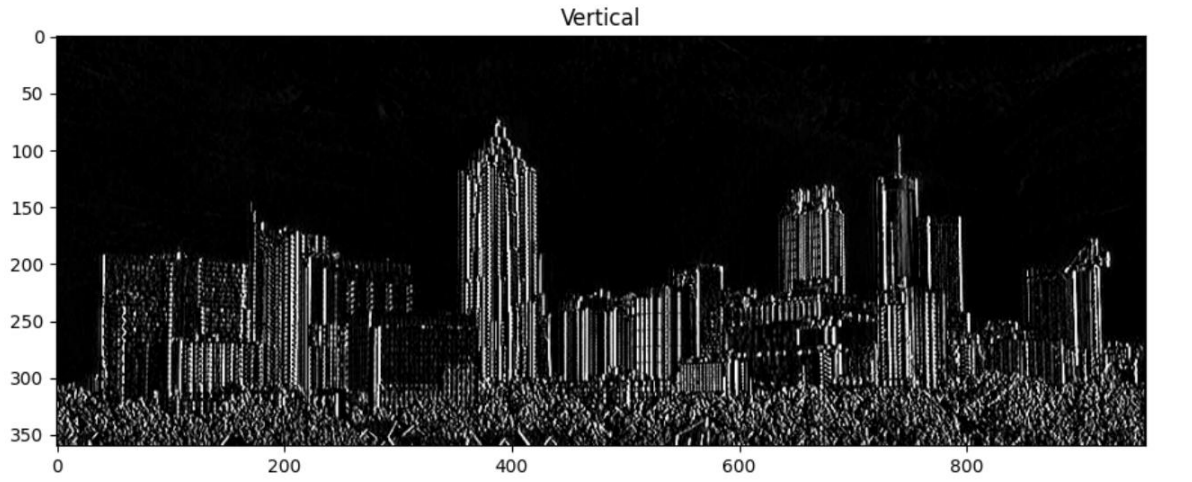
Summer 2025

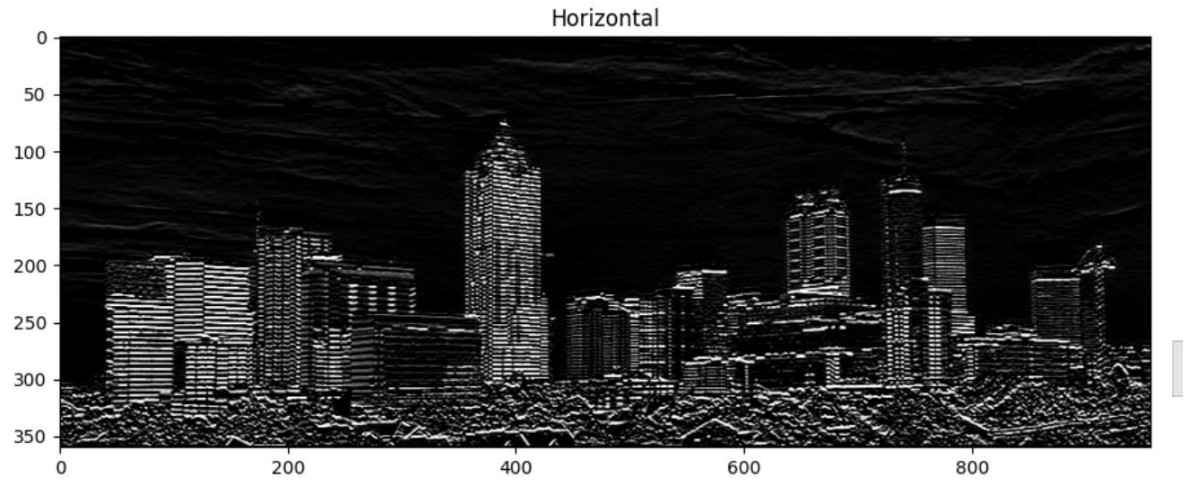
Bo Shen

Assignment 2; Part 1

Sobel Operator Edge Detector

Using the Sobel operators defined in the assignment prompt, the images below reflect the changes to the image with highlighted vertical and horizontal edges

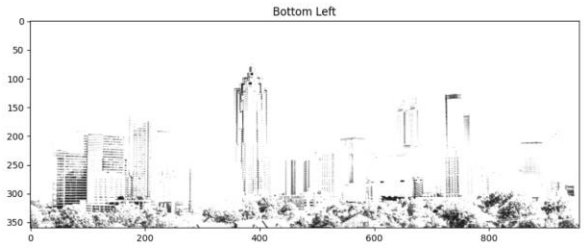


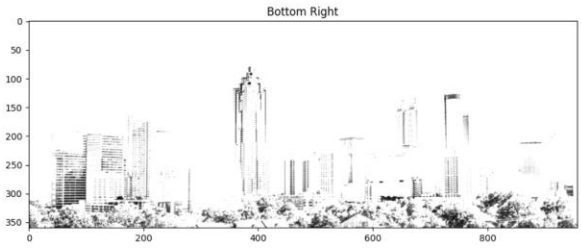


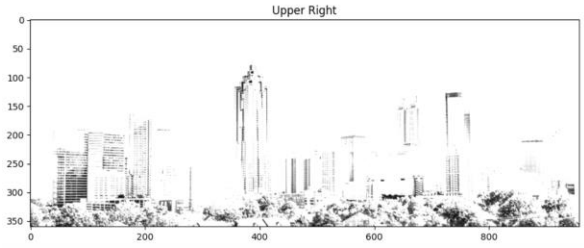
Corner Detection

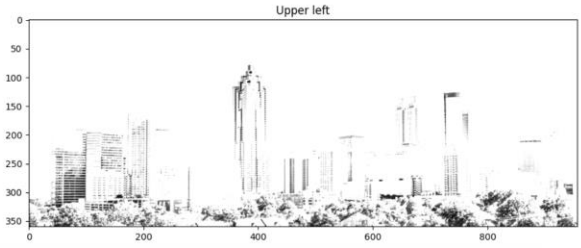
Surprisingly, the effect of the following corner detection kernels, which were given in the lecture slides, did not produce visually significant changes to the image.

```
# corner detection kernels

bottomLeft = np.array([[ 0, 1, 0],
                       [  0,  1,  1],
                       [  0,  0,  0]])

bottomRight = np.array([[ 0, 1, 0],
                        [ 1, 1, 0],
                        [ 0, 0, 0]])

upperRight = np.array([[ 0, 0, 0],
                       [ 1,  1, 0],
                       [  0,  1, 0]])

upperLeft = np.array([[ 0, 0, 0],
                      [ 0,  1, 1],
                      [ 0,  1,  0]])

fig = plt.figure(figsize=(24,24))
```









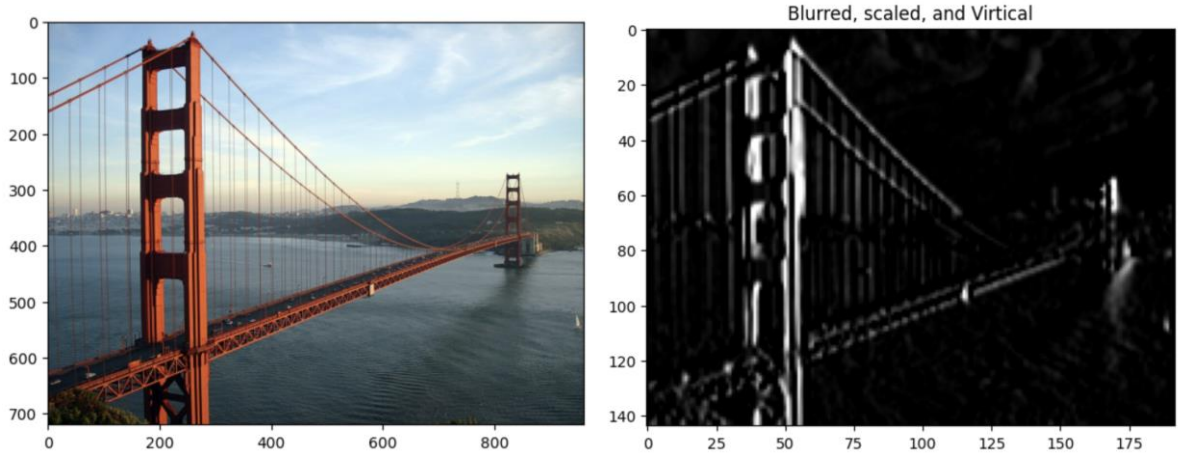
Scaling/Blurring

Here, I am dividing each neuron in the 2x2 kernel by the total amount of neurons in the kernel in order to average out each neuron in the image. This produces the blurring effect since the value of each pixel is changed to the average of the entire kernel. In the next line, we are

producing the scaling effect by specifying that we only want every 50th column and every 50th row from the original image to appear in the scaled down version.

```
blurred_image = cv2.filter2D(gray, -1, S2x2/4.0)

shrink = blurred_image[::50, ::50]
```

Challenge



Here I demonstrate a blurred, scaled, and vertically filtered image. Using a 10x10 kernel to blur, choosing every 5 pixel to scale down, and applying the vertical Sobel operator edge detector (kx), I am able to transform my custom image.

```
S10x10 = np.array([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                   [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                   [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                   [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                   [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                   [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                   [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                   [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                   [1, 1, 1, 1, 1, 1, 1, 1, 1, 1],
                   [1, 1, 1, 1, 1, 1, 1, 1, 1, 1]])

# Filter the image using filter2D, which has inputs: (grayscale imag
blurred_image = cv2.filter2D(gray3, -1, S10x10/100.0) # bluring

shrink = blurred_image[::5, ::5] #scaling down

virt_image = cv2.filter2D(shrink, -1, kx) #using sobel virt kernel
```